

Java – Beautifully Broken

Java learns to read your key files, and Quarkus 4 raises the floor to Java 21

Issue #4

Thu, 08 Oct 2026

~35 min read

Level 3/5

Today's two stories

1. **JEP 542 makes the PEM API final in JDK 28.** After three preview rounds, Java gets a standard, built-in way to turn the `-----BEGIN PRIVATE KEY-----` files that every certificate tool produces into real `PrivateKey` and `X509Certificate` objects, and back. No Bouncy Castle, no hand-rolled Base64 stripping, no keystore conversion step.
2. **Quarkus 4.0 reaches its first Beta, with the final release due at the end of November.** Red Hat's cloud-native framework moves to a Java 21 minimum, Jackson 3, Vert.x 5, Hibernate ORM 8 and HTTP/3, while Quarkus 3.40 becomes the LTS line that's supported until September 2027.
3. **On the radar:** 4 more things that happened, listed at the end.

STORY 1 JEP 542: Java finally reads PEM files on its own

JDK 28 · OPENJDK · SECURITY-LIBS · TARGETED TO JDK 28 (6 OCT 2026) · ~15 MIN READ

Ask anyone who has wired up mutual TLS in a Java service what the most annoying step was, and there's a good chance the answer isn't the cryptography. It's the file format. Your certificate authority, OpenSSL and your Kubernetes secret all hand you neat little text files that start with `-----BEGIN`. The JDK, for its entire history, has looked at those files and shrugged. This week that officially changed: the API that teaches Java to read and write them is now final for JDK 28.

WHAT HAPPENED

On 6 October, Oracle's Inside.java site announced that **JEP 542, "PEM Encodings of Cryptographic Objects", is targeted to JDK 28.**¹ InfoQ had reported in mid-August that the JEP was Proposed to Target, with its review due to end on 26 August, and summarised its purpose as finalising the feature "after three rounds of preview delivered in JDK 25 through JDK 27."^{2,3}

The most important sentence in the JEP is a short one. As quoted by JVM Weekly, it says: "We here propose to finalize the API without further change." In other words, code written against the third preview in JDK 27 should compile unchanged on JDK 28, just without the `--enable-preview` flag.⁴

JDK 28 is a non-LTS feature release scheduled for March 2027.⁵ So nothing here is production-ready today on a final JDK. But the API is now settled, and for once that's the whole story: it's not going to move again.

THE BACKGROUND YOU NEED

What a PEM file actually is

Cryptographic objects like keys and certificates are, underneath, binary data. The standard way to lay that data out is **DER** *≈ Distinguished Encoding Rules: a strict binary format for structured data, used for keys and certificates*. Binary data is awkward to paste into an email, a YAML file or an environment variable, so someone long ago had the idea of wrapping it in text.

That idea is **PEM** *≈ Privacy-Enhanced Mail: a text wrapper around binary cryptographic data, with BEGIN/END lines around Base64 content*. The name is a historical accident. RFC 7468, published in April 2015 by S. Josefsson and S. Leonard, explains that "the tradition within the RFC series can be traced back to Privacy-Enhanced Mail (PEM)" in RFC 1421, an email-security standard that never really took off. The text format it used, however, became the default way to move keys and certificates around.⁶

A PEM file looks like this:

```
-----BEGIN CERTIFICATE-----
MIIDdzCCA1+gAwIBAgIEAgAAuTANBgkqhkiG9w0BAQUF
...more Base64, 64 characters per line...
-----END CERTIFICATE-----
```

The middle part is **Base64** *≈ a way of writing arbitrary bytes using only 64 safe text characters*. The word after **BEGIN** is the label, and it tells you what's inside. RFC 7468 standardised the labels that tools had been using for years: **CERTIFICATE**, **X509 CRL**, **CERTIFICATE REQUEST**, **PRIVATE KEY**, **ENCRYPTED PRIVATE KEY** and **PUBLIC KEY**, among others. It also says labels must match on the **BEGIN** and **END** lines, lines should be exactly 64 characters, and that "parsers MUST NOT malfunction" when there's other text before the **BEGIN** line, because many tools print a human-readable summary of a certificate above it.⁶

What's inside a PEM private key file

PEM text

-----BEGIN PRIVATE KEY----- ... -----END

Base64 characters

64 per line, text-safe

DER bytes

binary layout defined by PKCS #8

PrivateKey object

what your Java code actually wants

Each layer is an encoding of the one below. Format rules from RFC 7468.

Two more standards decide what the DER bytes mean. **PKCS #8** *≈ the standard binary format for private keys, optionally password-encrypted* covers private keys. **X.509** *≈ the standard format for certificates, public keys and certificate*

revocation lists covers certificates, public keys and **CRLs** *≈ certificate revocation lists: signed lists of certificates that are no longer trusted* .

How Java has handled this until now

Java has always been happy with DER. Its own key-handling classes, such as `PKCS8EncodedKeySpec` and `X509EncodedKeySpec` , take the binary bytes. What Java never had was the thin text layer on top. JEP 470, the first preview, lists the consequences plainly: encoding a public key is "straightforward but tedious", decoding a PEM key means parsing the text, picking the right factory and working out the algorithm yourself, and encrypting or decrypting a private key "takes more than a dozen lines of code". It notes that PEM is used everywhere: by certificate authorities, OpenSSL, OpenSSH and hardware keys such as YubiKeys. And it cites an April 2022 survey of Java cryptography users that confirmed the pain.⁷

So Java developers have lived with three workarounds.

The first is **converting everything into a keystore**. A **keystore** *≈ a password-protected file that holds keys and certificates in a Java-friendly container* comes in two common flavours: **JKS** *≈ Java KeyStore, an older Java-only keystore format* and **PKCS #12** *≈ an industry-standard keystore format, the default in Java since JDK 9* . A VirtusLab walkthrough of a mutual-TLS client shows the typical dance: use `openssl` to bundle a certificate and key into PKCS #12, use `keytool` to import the CA certificate into a truststore, and only then load them in Java. The author calls the extra step a source of deployment complexity and a dependency on external tools.⁸

The second is **Bouncy Castle**, the third-party cryptography library whose `PEMParser` reads almost anything. It works well, but it's another dependency you must keep patched and compatible.⁸

The third is **letting your framework hide it**. Spring Boot has accepted PEM files for embedded web servers since 2.7, and in 3.1 (June 2023) introduced **SSL bundles** *≈ Spring Boot's named, reusable sets of SSL keys and certificates configured in properties* with a dedicated `spring.ssl.bundle.pem` section, precisely because, in the Spring team's words, trust material comes in several formats and Java's connection APIs each expect different objects.⁹

Kafka (via KIP-651) and Netty's `SslContextBuilder` grew their own PEM support too.⁸ Each framework solved the problem again, separately.

Here's the "do it yourself" version that countless codebases contain, for an unencrypted RSA private key. It's short, and it's fragile: it only works for one label, one algorithm and no encryption.

```
// Works on any JDK. Only handles an unencrypted RSA PKCS #8 key.
String b64 = pem
    .replace("-----BEGIN PRIVATE KEY-----", "")
    .replace("-----END PRIVATE KEY-----", "")
    .replaceAll("\\s", "");
byte[] der = Base64.getDecoder().decode(b64);
var spec = new PKCS8EncodedKeySpec(der);
PrivateKey key = KeyFactory.getInstance("RSA").generatePrivate(spec);
```

(We compiled and ran this on JDK 21 against a freshly generated RSA key; it round-trips correctly.) Notice the hard-coded `"RSA"` . Hand it an elliptic-curve key, or an encrypted one, and it breaks.

Three previews and a last-minute change of plan

The PEM API took an unusually winding road for such a small feature. JEP 470, created in January 2023 and owned by Anthony Scarpino, was delivered as a **preview API** *≈ a complete, but not yet permanent, API that you can only use with the `--enable-preview` flag, so it can still change* in JDK 25.^{7,10} JEP 524 brought a second preview in JDK 26 to gather more experience.^{11,10}

The plan was then to finalise it in JDK 27. A JEP draft from February 2026 says exactly that, and lists the clean-ups that would go with it.¹⁰ Instead, InfoQ reported in June that "due to last-minute feedback, it was decided to deliver a third preview", as JEP 538.¹² JVM Weekly quotes Mark Reinhold's note on the `jdk-dev` list: "Due to late-breaking feedback, the owner of this JEP has decided do one more preview round." The feedback itself was never made public.⁴ The changes still went in, and JEP 542 now finalises that third-preview API as is.

The PEM API's road to final



Sources: RFC 7468; JEP 470; JEP 538 draft history; InfoQ; Inside.java.

HOW IT WORKS

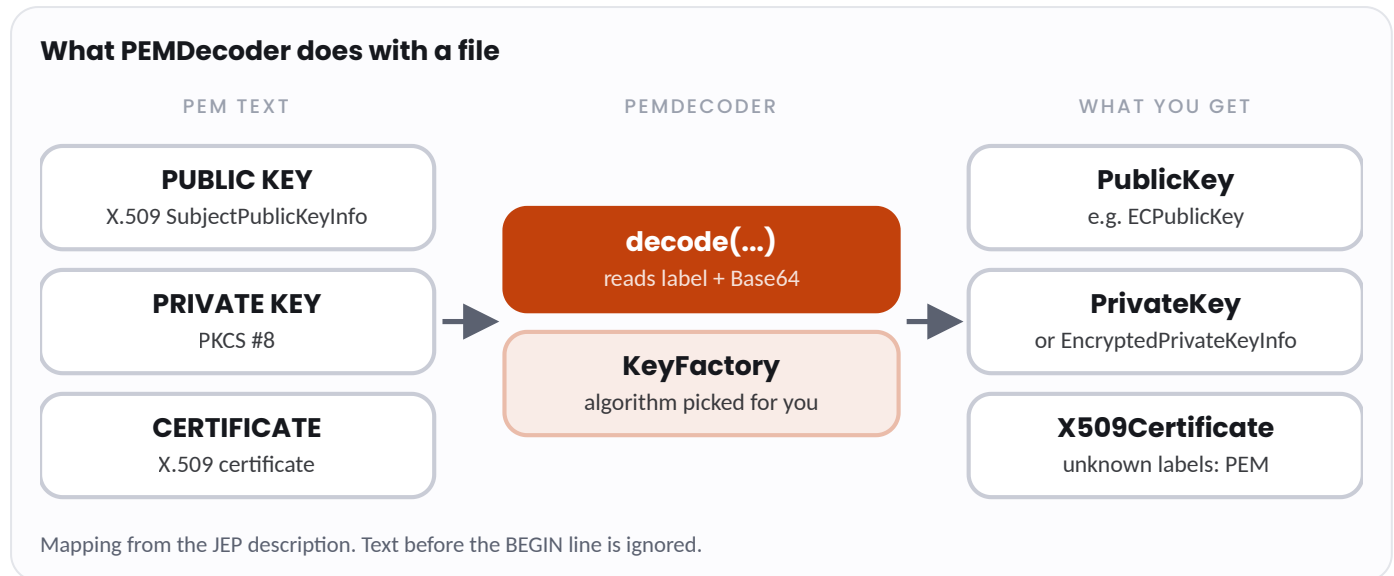
One interface, two workers

The whole API lives in `java.security` and has four main pieces.¹⁰

- **BinaryEncodable** is a **sealed interface** *≈ an interface that lists exactly which types may implement it, so the compiler knows every possible case (Java 17+)* with no methods at all. It's a label for "a Java type that has a standard binary encoding". The types it permits are `AsymmetricKey`, `KeyPair`, `PKCS8EncodedKeySpec`, `X509EncodedKeySpec`, `EncryptedPrivateKeyInfo`, `X509Certificate`, `X509CRL` and a new class, `PEM`.
- **PEMEncoder** turns any of those objects into PEM text.
- **PEMDecoder** turns PEM text back into objects.
- **PEM** is a catch-all holder for PEM content that has no Java type, such as a certificate signing request.

The interface was called `DEREncodable` in the first two previews and was renamed to `BinaryEncodable` in the third, to describe the data more accurately. `PEM` was originally a **record** *≈ a compact Java class that just carries data, declared with the `record` keyword (Java 16+)* called `PEMRecord`; it became an ordinary class that stores its content in binary form and copies it defensively.^{10,12} If you tried the JDK 25 preview, those are the two renames that will break your code.

Encoders and decoders are **immutable** *≈ cannot be changed after they're created, so they're safe to share* and **thread-safe** *≈ safe to use from several threads at once without extra locking*. You get one with `of()` and derive configured copies with `with...` methods, the same style the JDK uses for `HexFormat` and `Base64`'s encoders, which the JEP names as its model.¹⁰



Decoding: let the type system do the sorting

The decoder figures out the algorithm from the key data itself, so the hard-coded `"RSA"` disappears. You can ask for a specific type, or take whatever comes back and use **pattern matching for switch** *≈ a switch that branches on a value's type and binds it to a variable (Java 21+)*. These are the JEP's own examples:¹⁰

Load a private key from PEM text

Today: by hand

```
String b64 = pem
    .replace("-----BEGIN PRIVATE KEY-----", "")
    .replace("-----END PRIVATE KEY-----", "")
    .replaceAll("\\s", "");
var spec = new PKCS8EncodedKeySpec(
    Base64.getDecoder().decode(b64));
PrivateKey key = KeyFactory
    .getInstance("RSA") // guess!
    .generatePrivate(spec);
```

JDK 28: JEP 542

```
PEMDecoder pd = PEMDecoder.of();

// ask for exactly what you expect
ECPrivateKey key =
    pd.decode(pem, ECPrivateKey.class);

// or decrypt on the way in
ECPrivateKey k2 = pd
    .withDecryption(password)
    .decode(pem, ECPrivateKey.class);
```

Left compiled on JDK 21. Right is from the JEP text; JDK 28 isn't out yet, so we couldn't compile it.

```
// JDK 28 (JDK 25–27 with --enable-preview), from the JEP
switch (PEMDecoder.of().decode(pem)) {
    case PublicKey publicKey -> ...;
    case PrivateKey privateKey -> ...;
    default -> throw new IllegalArgumentException(...);
}
```

A few rules are worth knowing. If you ask for a class and the file holds something else, you get a `ClassCastException`. Malformed text throws `IllegalArgumentException`. Text before the BEGIN line is skipped, as RFC 7468 requires, unless you decode into a `PEM` object, which keeps it as "leading data". There's also `withFactory(provider)` if you need a specific **security provider** *≈ a pluggable implementation of cryptographic algorithms, e.g. the JDK's built-in one or Bouncy Castle* to build the key objects.¹⁰

Encoding and encryption

Writing PEM is the mirror image:

```
// JDK 28, from the JEP
PEMEncoder pe = PEMEncoder.of();
String pem = pe.encodeToString(new KeyPair(publicKey, privateKey));
String locked = pe.withEncryption(password).encodeToString(privateKey);
```

`encode` returns bytes in the ISO-8859-1 character set, and `encodeToString` returns a `String`. `withEncryption` produces an `ENCRYPTED PRIVATE KEY` block, using **password-based encryption** *≈ encryption where the key is derived from a password, so a human can unlock the file*. The default algorithm comes from the `jdk.epkcs8.defaultAlgorithm` security property, which defaults to `PBEWithHmacSHA256AndAES_128`. Because the algorithm and its parameters are recorded inside the encrypted file, a future change of default won't stop old files from opening.¹⁰

If you need control over the algorithm or provider, the existing `EncryptedPrivateKeyInfo` class gains static `encrypt(...)` methods (renamed from `encryptKey` in the first preview, and now accepting a `KeyPair` or `PKCS8EncodedKeySpec` as well as a `PrivateKey`), plus `getKey(...)` and `getKeyPair(...)` to decrypt. Decrypt a file without a password and you get the still-encrypted `EncryptedPrivateKeyInfo` back, to unlock later.¹⁰

Finally, there's a new `CryptoException`, an **unchecked exception** *≈ an exception that extends RuntimeException, so the compiler doesn't force you to catch or declare it* for cryptographic failures, such as a wrong password, in places where Java's traditional checked `GeneralSecurityException` would make the API clumsy.¹⁰

What it deliberately doesn't do

The JEP's goals are ease of use and the standard formats: PKCS #8 for private keys, X.509 for public keys, certificates and CRLs, and PKCS #8 v2.0 for encrypted keys.¹⁰ Formats without a Java API, like a `CERTIFICATE REQUEST`, come back as a raw `PEM` object with a `type()` and `content()`, so you can at least handle them

yourself. The JEP also explains why it didn't bolt PEM onto `KeyFactory` or `EncodedKeySpec` : providers support key types inconsistently, and a key spec can't represent a certificate.^{10,13}

WHAT IT MEANS FOR YOU

If you're on Spring Boot, nothing changes tomorrow. SSL bundles already accept PEM files, and Spring will keep doing the work for you.⁹ Where you'll notice JEP 542 is in the code that sits *outside* the framework: a CLI that signs something, a test that spins up a TLS server, a Kubernetes operator that reads a mounted secret, or a small service that never wanted Bouncy Castle in its dependency tree. For all of those, building an `SSLContext` straight from PEM files becomes a few lines of standard Java. VirtusLab's mutual-TLS example against an Nginx server shows exactly that: the keystore conversion step disappears.⁸

Library maintainers have a reason to drop a dependency, eventually. The tooling world has already noticed. On 27 September, the maintainer of the Modernizer Maven plugin, which flags legacy APIs that have modern JDK replacements, opened an issue proposing new JDK 28 rules: flag Bouncy Castle's `PemReader` and `PEMParser` and suggest `PEMDecoder`, flag `PemWriter` and `JcaPEMWriter` and suggest `PEMEncoder`.¹⁴ Expect static-analysis tools to start nudging you once JDK 28 ships.

But read the caveats in that same issue before ripping Bouncy Castle out. It points out that Bouncy Castle's parser handles formats the JDK API doesn't map to Java objects, such as the older "SEC1" elliptic-curve key files and OpenSSL's legacy encrypted-key format, and that it returns its own holder objects where the JDK returns standard Java types.¹⁴ If your keys come from older tooling, check the label on the first line before you migrate. `-----BEGIN PRIVATE KEY-----` is PKCS #8 and fine. `-----BEGIN EC PRIVATE KEY-----` or `-----BEGIN RSA PRIVATE KEY-----` are older formats, and per the Modernizer issue they come back as a raw `PEM` object rather than a key. You can convert them once with `openssl pkcs8 -topk8`.

And the LTS math matters. JDK 28 isn't an LTS release. If your production fleet only moves between LTS versions, the first LTS with the final API is JDK 29, scheduled for September 2027. JDK 25, the current LTS, only has the *first* preview, with the old names (`DEREncodable`, `PEMRecord`, `encryptKey`).^{7,10} Code written for that preview won't compile against the final API, so don't ship it.

🌟 Watch out for

- **Preview names differ.** Code written for JDK 25's first preview uses `DEREncodable`, `PEMRecord` and `encryptKey`; JDK 28's final API uses `BinaryEncodable`, `PEM` and `encrypt`.¹⁰
- **Not every PEM file becomes a key.** Legacy OpenSSL formats (`EC PRIVATE KEY`, `RSA PRIVATE KEY`, old-style encrypted keys) aren't mapped to Java key objects.¹⁴ Convert them to PKCS #8 first.
- **Wrong type, runtime error.** `decode(pem, ECPrivateKey.class)` on an RSA key throws `ClassCastException`, not a compile error.¹⁰
- **Passwords are `char[]`, not `String`,** so you can wipe them from memory after use. Keep it that way in your own code.
- **It's encoding, not trust.** Decoding a certificate doesn't validate it. Chain building, expiry and revocation checks are still your `TrustManager`'s job.

WHAT HAPPENS NEXT

The implementation is going into JDK 28, whose general availability is scheduled for March 2027.⁵ Until then, you can try the API on JDK 27 with `--enable-preview`, and the code will carry over unchanged.⁴ The bigger milestone is September 2027's JDK 29 LTS, which will be the first long-term release where reading a `.pem` file needs nothing but the JDK. For a format that was invented for email in the early days of the internet, that's a long wait for a small API. It's also a nice example of what JEPs do well: three previews, a public argument about names, and a result that's boring in the best possible way.

Words from this story

DER: Distinguished Encoding Rules, a strict binary format for structured data used by keys and certificates.

PEM: Privacy-Enhanced Mail; today, a text wrapper with BEGIN/END lines around Base64-encoded binary data.

Base64: a way of writing arbitrary bytes using only 64 safe text characters.

PKCS #8: the standard binary format for private keys, optionally password-encrypted.

X.509: the standard format for certificates, public keys and CRLs.

CRL: certificate revocation list, a signed list of certificates that are no longer trusted.

Keystore: a password-protected file holding keys and certificates in a Java-friendly container.

JKS: Java KeyStore, an older Java-only keystore format.

PKCS #12: an industry-standard keystore format, Java's default since JDK 9.

SSL bundles: Spring Boot's named, reusable sets of SSL keys and certificates, configured in properties (Boot 3.1+).

Preview API: a complete but not yet permanent API, usable only with `--enable-preview`.

Sealed interface: an interface that lists exactly which types may implement it (Java 17+).

Record: a compact data-carrier class declared with the `record` keyword (Java 16+).

Immutable: cannot be changed after it's created.

Thread-safe: safe to use from several threads at once without extra locking.

Pattern matching for switch: a switch that branches on a value's type and binds it to a variable (Java 21+).

Security provider: a pluggable implementation of cryptographic algorithms.

Password-based encryption: encryption whose key is derived from a password.

Unchecked exception: an exception extending `RuntimeException`, which you don't have to catch or declare.

STORY 2 Quarkus 4.0 Beta: a new floor, a new engine room

QUARKUS · RED HAT · FRAMEWORKS · 4.0.0.BETA1 (1 OCT 2026), FINAL DUE END OF NOVEMBER · ~14 MIN READ

Major versions of a Java framework are rare, and they're where the bills come due. Quarkus hit 1.0 in 2019, and its last major, in 2023, was the great `javax` to `jakarta` rename that touched nearly every import in an enterprise codebase. The next one is now in beta, and while it won't rename your packages, it does replace almost everything underneath your code: the Java baseline, the networking engine, the JSON library and the persistence layer.

WHAT HAPPENED

On 1 October, Guillaume Smet announced **Quarkus 4.0.0.Beta1**, "the first Beta of the next major version". The final release is planned for the end of November, and the team asked extension developers to migrate and users to test their applications now.¹⁵

The headline changes, as listed in the announcement:¹⁵

- **Java 21** is the new minimum, which the post says makes virtual threads more central.
- **Netty 4.2 and Vert.x 5** form the new networking foundation.
- **QUIC** support brings opt-in **HTTP/3**, plus MQTT over QUIC and WebTransport.
- **Native transports** (epoll or io_uring), which also work in native executables.
- A new **Observation API** (`@Observed`) for metrics and traces with Micrometer and OpenTelemetry, and an Observability section in the Dev UI.
- **Jakarta REST 4**, with JSON Merge Patch and a new multipart API.
- **Hibernate ORM 8**, integrating Jakarta Persistence 4.0 and Jakarta Data 1.1.
- **Jackson 3**, a new major version where, in the post's words, everything changed except the annotations.
- **CDI 5.0**, a new JMS extension, and `@ExactlyOnce` for Kafka transactions.

One day earlier, on 30 September, the team shipped **Quarkus 3.40**, the new long-term-support release of the 3.x line.¹⁷ According to the releases page, 3.40 has community maintenance until 30 September 2027.¹⁸ That's the line most production teams will sit on while 4.x settles.

THE BACKGROUND YOU NEED

What Quarkus is, and why it exists

Quarkus is a Java framework for building services, in the same space as Spring Boot and Micronaut. It's open source with commercial builds and support from Red Hat and IBM, which the releases page lists alongside the community support dates.¹⁸ Its distinguishing idea is to do as much work as possible at **build time** rather than at start-up, which makes applications start fast and use less memory. That also makes it a natural fit for a **native executable** *≈ a program compiled ahead of time into a standalone binary for one operating system, which starts in milliseconds and doesn't need a separately installed JVM*, built with **GraalVM** *≈ Oracle's JDK distribution whose native-image*

tool compiles Java into native executables or **Mandrel** ≈ a downstream distribution of GraalVM focused on building native executables, used by default for Quarkus native builds .

Quarkus 1.0 Final shipped on 25 November 2019.¹⁹ Quarkus 3.0 followed on 26 April 2023, moving the framework from Jakarta EE 8 to **Jakarta EE 10** ≈ the 2022 version of the enterprise Java standards (persistence, REST, dependency injection and more), whose packages are named jakarta.*. The release notes put the pain bluntly: "the javax.* packages from Jakarta EE have been moved to jakarta.*," and Hibernate ORM moved from 5 to 6.2, "a new major release" with breaking changes. To soften the blow, Quarkus 3.0 introduced an update tool, **quarkus update**, that rewrites versions, packages and some configuration for you.²⁰

Between majors, Quarkus releases quickly. The releases page says new minor versions ship "every 4–6 weeks", and "LTS releases are published every 6 months" and "maintained for 12 months with critical bug fixes and security patches."¹⁸ An **LTS** ≈ long-term support: a release line that gets bug and security fixes for an extended period, so you don't have to upgrade every few weeks release is what most companies run in production. Right now the releases page lists 3.33 (supported until 25 March 2027) and 3.40 (until 30 September 2027).¹⁸



The engine room: Vert.x and Netty

Most Quarkus users write ordinary blocking code with annotations and never think about what's underneath. But everything sits on two libraries. **Netty** ≈ a widely used Java library for fast, low-level network I/O handles the sockets. **Vert.x** ≈ an Eclipse toolkit for building non-blocking (reactive) applications, built on Netty provides the HTTP server, the **event loop** ≈ a small number of threads that each handle many connections by never blocking, instead reacting to events as data arrives and the asynchronous programming model. When your REST endpoint answers a request, Vert.x accepted it, and Netty read the bytes.

Virtual threads, and why Quarkus 3 couldn't fully use them

Virtual threads ≈ lightweight threads managed by the JVM rather than the operating system, so you can have millions of them and write simple blocking code that still scales (final in Java 21) arrived as a final feature in Java 21. For a framework, they're a big deal: they let you keep the easy, blocking programming style while getting much of the scalability that used to require reactive code.

The catch was that Quarkus 3 had to keep running on Java 17, where virtual threads don't exist. Clement Escoffier explained the result in a September post on the quarkus.io blog: "Quarkus 3 already supported them, but through reflection hacks to avoid depending on the Java 21 API." He calls that approach hard to maintain and sub-optimal, and says virtual threads "felt like second-class citizens," because not every extension was tested against them.²¹

HOW IT WORKS

A higher floor: Java 21

The Java 21 minimum is the change that affects everyone, and Escoffier's post gives four reasons for it.²¹

1. **Virtual threads become first-class.** With Java 21 as the baseline there's "no reflection, no conditional paths," so each extension can be tested with virtual threads directly.
2. **The ecosystem is moving.** Jakarta EE 12 sets Java 21 as its baseline, and tools such as Error Prone and Apache Jena 6 have moved beyond Java 17. Starting Quarkus 4 on 17 would force "a mid-lifecycle bump."
3. **Support timelines.** Java 17 support ends in 2027 or 2029 depending on your vendor, and Quarkus 4 will live well past that.
4. **The test matrix.** Keeping 17 alongside 21, 25 and the upcoming Java 29 would be "not a sustainable number of variants."

Why not jump straight to Java 25? The post says Java 21 usage is still strong, and Quarkus already supports Java 25 features such as the Foreign Function & Memory API without depending on them. Native builds use Mandrel 25 regardless.²¹ (Quarkus already made GraalVM/Mandrel 25 the minimum for native builds back in 3.36.²²)

The release notes for Beta1 show what "first-class" means in practice: the preview-era reflection for virtual threads has been removed.¹⁶

A new engine room: Vert.x 5 and Netty 4.2

Quarkus 4 moves to Vert.x 5 (Beta1 ships 5.1.8) on top of Netty 4.2.^{16,15} If you only write annotated REST endpoints, you mostly won't see this. If you, or an extension you use, talk to Vert.x directly, you will. The Vert.x 5 migration guide lists the big changes: it "only retains the future model and thus the callback model is gone", Netty types are no longer part of the public API, many options classes give way to builders, and a long list of 4.x deprecations is removed.²³

The Quarkus 4 stack, top to bottom

Your code

REST endpoints, entities, @Observed

Quarkus extensions

REST 4, CDI 5 (ArC), Hibernate ORM 8

Jackson 3

tools.jackson.*, JSON in and out

Vert.x 5

event loop, futures-only API

Netty 4.2

HTTP/1.1, HTTP/2, opt-in HTTP/3

Java 21+ and the OS

virtual threads; epoll / io_uring

Component versions from the 4.0.0.Beta1 announcement and release notes.

The new networking layer is also what brings **QUIC** *≈ a transport protocol that runs over UDP, with encryption built in and faster connection set-up than TCP*, the foundation of **HTTP/3** *≈ the newest version of HTTP, which runs over QUIC instead of TCP*. It's opt-in, and in dev and test mode Quarkus generates the certificates it needs automatically.¹⁵ Optional native transports use Linux's **epoll** *≈ the Linux system call interface for efficiently watching many network connections at once* or the newer **io_uring** *≈ a Linux interface that lets programs submit and complete I/O through shared queues with fewer system calls* instead of Java's generic networking code.¹⁵

Jackson 3: the change you'll actually feel

Jackson *≈ a very widely used JSON library for Java; it turns JSON into Java objects and back* is underneath every JSON request and response in a typical Quarkus or Spring service. Jackson 3.0, a new major version with deliberate breaking changes, was released on 3 October 2025.²⁴ Quarkus 4 Beta1 ships Jackson 3.1.4.¹⁶

The big changes, from the Jackson team's own announcement and migration guide:^{24,25}

- **New packages and Maven coordinates.** `com.fasterxml.jackson` becomes `tools.jackson`, deliberately, so that Jackson 2 and 3 can coexist on one **classpath** *≈ the list of JARs and directories the JVM loads classes from*. The big exception is `jackson-annotations: @JsonProperty`, `@JsonIgnore` and friends keep their old package, which is why the Quarkus post says everything changed except the annotations.
- **Java 17 baseline**, up from Java 8.
- **Unchecked exceptions.** The root is now `JacksonException`, which extends `RuntimeException` instead of `IOException`. `JsonProcessingException` is gone, and `JsonMappingException` becomes `DatabindException`.
- **Immutable mappers.** You configure an `ObjectMapper` once, through a builder, and derive variants with `rebuild()`. `setConfig()` and `copy()` are removed.

- **New defaults.** `FAIL_ON_UNKNOWN_PROPERTIES` is now off. `WRITE_DATES_AS_TIMESTAMPS` is off too, so dates come out as ISO-8601 text by default. The default time zone is UTC, not the JVM's. And the Java 8 date/time module is now built in.

Configuring a mapper: Jackson 2 vs Jackson 3

Jackson 2 (Quarkus 3)

```
import com.fasterxml.jackson
    .databind.ObjectMapper;

ObjectMapper m = new ObjectMapper();
m.registerModule(new JavaTimeModule());
m.enable(
    SerializationFeature.INDENT_OUTPUT);
// checked IOException
m.readValue(json, Order.class);
```

Jackson 3 (Quarkus 4)

```
import tools.jackson
    .databind.json.JsonMapper;

JsonMapper m = JsonMapper.builder()
    // java.time support is built in
    .enable(
        SerializationFeature.INDENT_OUTPUT)
    .build(); // immutable from here
// unchecked JacksonException
m.readValue(json, Order.class);
```

Illustrative, based on the Jackson 3 migration guide. In Quarkus you'd normally customise the injected mapper rather than build your own.

Quarkus isn't alone. Spring Framework 7 also defaults to Jackson 3 "in our entire stack", with Jackson 2 deprecated and scheduled for removal in Spring Framework 7.2.²⁶ If you work in both worlds, it's the same migration twice.

Persistence, observability and the long tail

Hibernate ORM 8 brings an implementation of **Jakarta Persistence** *≈ the standard Java API for mapping objects to relational database tables, formerly JPA* 4.0 and **Jakarta Data** *≈ a newer standard for repository-style data access, where you declare an interface and the implementation is generated* 1.1.¹⁵ Hibernate ORM 8.0 itself is still in beta: InfoQ reported its second and third betas in late September, highlighting version locking for `StatelessSession`.²⁷

Observability defaults change. In **OpenTelemetry** *≈ an open standard and set of libraries for collecting traces, metrics and logs from applications*, metrics and logs are now stable and on by default, the default export protocol moves to `http/protobuf`, and the default trace sampling rate becomes 10%.¹⁶ The new `@Observed` annotation records metrics and traces through one API that works with **Micrometer** *≈ a metrics library for Java that exports to Prometheus, Datadog and other back ends* and OpenTelemetry.¹⁵

And then there's the long tail of a major release. Beta1 bans **dynamic proxies** *≈ classes generated at run time to intercept method calls, via `java.lang.reflect.Proxy`*, requires the management port to differ from the main HTTP port, needs Gradle 9.6 for the Quarkus Gradle plugin, drops the 3.x artifact relocations and removes deprecated configuration properties from before 3.33. Legacy `javax.ws.rs` annotations now produce warnings, the Scala extension is gone and `H2DatabaseTestResource` is deprecated in favour of Dev Services.¹⁶

WHAT IT MEANS FOR YOU

If you're on Java 21 or newer, the floor isn't your problem. In Escoffier's words: "If you're on 21+, nothing changes." **If you're on 17**, he says "now is a good time to plan the move."²¹ The good news is that you don't have to plan it this month. Upgrade to **3.40 LTS**, which still runs on Java 17 and is supported until September 2027, and treat that year as your window to move both the JDK and the framework.^{18,22} The 3.40 announcement says `quarkus update` works from any version, even 2.x, and handles most changes, though some still need manual work.¹⁷

Your JSON output may change without a compile error. This is the one to test carefully. If any client depends on dates being numeric timestamps, on unknown properties being rejected, or on the JVM's time zone, Jackson 3's new defaults will change behaviour quietly.²⁵ Snapshot tests of your API responses are the cheapest insurance. Watch for libraries that still drag in Jackson 2: because the packages differ, both versions can sit on the classpath at once, which avoids crashes but can mean your custom serialiser is registered with the wrong one.

Check your telemetry bill and your dashboards. A default of 10% trace sampling and metrics and logs switched on by default can change both what you see and what you pay for, in either direction.¹⁶ Set the values explicitly rather than inheriting the new defaults by surprise.

If you maintain an extension or call Vert.x directly, start now. The beta exists for you. The announcement points extension developers to a migration guide on the Quarkus wiki, describes it as denser than usual, and invites questions on the project's Zulip chat.¹⁵ The Vert.x 5 move from callbacks to futures is the kind of change that touches a lot of lines.²³

In the real world

Ecosystem projects are already lining up behind 3.40 LTS. JobRunr 9 shipped with day-one support for it, and the 3.40 platform bumps Camel Quarkus and Quarkus CXF to 3.40.0, Quarkus LangChain4j to 1.13.3 and the Quarkus MCP Server to 2.0.1.¹⁷ Security fixes also keep flowing into the 3.x lines: in September, Quarkus 3.39.5 resolved 32 CVEs across its ecosystem, with the fixes backported to the 3.33 and 3.37 trains.²⁷ For the next year, 3.x is where those fixes will land for most people.

☀ Watch out for

- **It's a beta.** Don't run 4.0.0.Beta1 in production. The final release is planned for the end of November, and details can still change.¹⁵
- **Jackson's silent changes.** Dates as ISO strings, unknown properties allowed, UTC as the default time zone: these change behaviour without failing to compile.²⁵
- **Native builds need Mandrel/GraalVM 25.** That's already true since 3.36, and Quarkus 4 moves native metadata to the newer `reachability-metadata.json` format.^{22,16}
- **Removed deprecations.** Config properties deprecated before 3.33 are gone, so an app that still uses them will behave differently or fail at start-up.¹⁶ Clear deprecation warnings on 3.40 first.
- **Gradle users need 9.6+** for the Quarkus plugin.¹⁶

WHAT HAPPENS NEXT

The final 4.0 release is planned for the end of November.¹⁵ Before then, expect more betas or release candidates and a growing migration guide. After that, the usual Quarkus rhythm should resume: a minor release every few weeks and a new LTS every six months, with 3.40 overlapping until September 2027.¹⁸ The bigger picture is that the Java framework world is converging on the same new floor at once. Jakarta EE 12 sets Java 21 as its baseline,²¹ Spring and Quarkus both moved to Jackson 3, and virtual threads are no longer something frameworks have to sneak in through reflection. If your services are still on Java 17, 2027 is the year that stops being comfortable.

Words from this story

Native executable: a program compiled ahead of time into a standalone binary that starts in milliseconds.

GraalVM: Oracle's JDK distribution whose native-image tool compiles Java into native executables.

Mandrel: a downstream distribution of GraalVM focused on native executables, used for Quarkus native builds.

Jakarta EE 10: the 2022 version of the enterprise Java standards, with `jakarta.*` packages.

LTS: long-term support, a release line that gets fixes for an extended period.

Netty: a widely used Java library for fast, low-level network I/O.

Vert.x: an Eclipse toolkit for non-blocking (reactive) applications, built on Netty.

Event loop: a few threads that each handle many connections by never blocking.

Virtual threads: lightweight JVM-managed threads that let blocking code scale (final in Java 21).

QUIC: a UDP-based transport protocol with built-in encryption and fast connection set-up.

HTTP/3: the newest version of HTTP, running over QUIC.

epoll: the Linux interface for efficiently watching many connections at once.

io_uring: a Linux interface for submitting and completing I/O through shared queues.

Jackson: a very widely used JSON library for Java.

Classpath: the list of JARs and directories the JVM loads classes from.

Jakarta Persistence: the standard API for mapping objects to database tables (formerly JPA).

Jakarta Data: a standard for repository-style data access with generated implementations.

OpenTelemetry: an open standard for collecting traces, metrics and logs.

Micrometer: a metrics library for Java that exports to many monitoring back ends.

Dynamic proxies: classes generated at run time to intercept method calls.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Quarkus Desktop:** a new extension lets you build AWT and Swing desktop apps with CDI beans and compile them to native executables for Windows, Linux and macOS.²⁸
- **JetBrains Air comes to the IDEs:** an Early Access Program puts JetBrains' multi-agent tool window, which runs Codex, Copilot, Junie, Cursor and other ACP agents side by side in temporary git worktrees, inside the 2026.3 EAP builds.²⁹
- **Agent Helidon at JavaOne:** an Inside.java session argues that AI agent back ends need the same scalability focus as any service, showing virtual-thread MCP servers and Helidon's compile-time LangChain4j integration.³⁰
- **Quarkus 3.39.5 fixes 32 CVEs:** the September maintenance release resolved 32 vulnerabilities across the Quarkus ecosystem and backported the fixes to the 3.33 and 3.37 trains.²⁷

1. **Inside Java, JDK 28 tag:** "JEP targeted to JDK 28: 542: PEM Encodings of Cryptographic Objects", 6 Oct 2026 (checked 8 Oct 2026).

2. **Java News Roundup: JDK 27-RC1, OpenJDK JEPs, Jakarta EE, BellSoft, Helidon, Micrometer, Tika 4.0,** InfoQ, 25 Aug 2026.

3. **Java News Roundup: GraalVM, Jakarta Data, JNoSQL, Azul Payara, WildFly, Quarkus, Atmosphere,** InfoQ, 31 Aug 2026.

4. **JDK 27 is here,** Artur Skowronski, JVM Weekly vol. 192.

5. **JDK 27 and JDK 28: What We Know So Far,** InfoQ, 24 Aug 2026.

6. **RFC 7468: Textual Encodings of PKIX, PKCS, and CMS Structures,** S. Josefsson and S. Leonard, IETF, April 2015.

7. **JEP 470: PEM Encodings of Cryptographic Objects (Preview),** Anthony Scarpino, OpenJDK.

8. **JEP-538: Native PEM Support in Java**, Michał Zyga, VirtusLab, 18 Jun 2026.
9. **Securing Spring Boot Applications With SSL**, Scott Frederick, spring.io, 7 Jun 2023.
10. **JEP draft: PEM Encodings of Cryptographic Objects (JDK-8376991, delivered as JEP 538)**, Anthony Scarpino, OpenJDK, updated Feb 2026. The JEP 542 page itself returned 403 from our sandbox; JVM Weekly and InfoQ confirm JEP 542 finalises this API without further change.
11. **New candidate JEP: 524: PEM Encodings of Cryptographic Objects (Second Preview)**, Mark Reinhold, jdk-dev, 11 Sep 2025.
12. **Java News Roundup (week of 1 Jun 2026)**, InfoQ, 8 Jun 2026.
13. **JEP 542: PEM Encodings of Cryptographic Objects**, daily.dev summary of the Inside.java post, 6 Oct 2026.
14. **Add Java 28 PEM API violations for BouncyCastle PEM readers and writers (issue #448)**, gaul/modernizer-maven-plugin, 27 Sep 2026.
15. **Quarkus 4.0.0.Beta1 released**, Guillaume Smet, quarkus.io, 1 Oct 2026.
16. **quarkusio/quarkus 4.0.0.Beta1 release notes**, mirrored from GitHub.
17. **Quarkus 3.40 LTS - new LTS version**, Guillaume Smet, quarkus.io, 30 Sep 2026.
18. **Quarkus Releases**, quarkus.io (checked 8 Oct 2026).
19. **Quarkus 1.0.0.Final bits are here**, quarkus.io, 25 Nov 2019.
20. **Quarkus 3.0 Final released**, Guillaume Smet, quarkus.io, 26 Apr 2023.
21. **Java 21: the foundation for Quarkus 4**, Clement Escoffier, quarkus.io, 10 Sep 2026.
22. **Bumping the minimum officially supported GraalVM version to 25.0**, Foivos Zakkak, quarkus.io, 29 May 2026.
23. **Vert.x 5 migration guide**, Eclipse Vert.x.
24. **Jackson 3.0.0 (GA) released**, Tatu Saloranta, Oct 2025.
25. **Migrating to Jackson 3**, FasterXML/jackson on GitHub.
26. **Spring Framework 7.0.0-M5 available now**, spring.io, 15 May 2025.
27. **Java News Roundup: TornadoVM 7.0, Groovy 6.0, GraalVM, Hibernate, Quarkus, Gradle, Maven**, InfoQ, 28 Sep 2026.
28. **Build AWT and Swing desktop applications with Quarkus Desktop**, Fouad Almalki, quarkus.io, 30 Sep 2026.
29. **Air in JetBrains IDEs: Early Access Program**, Dominique Rolink and Denis Shiryayev, JetBrains, Oct 2026.
30. **Agent Helidon: License to Scale**, Ryan Eberhard, Inside.java, 1 Oct 2026.